

СИСТЕМА ПРОГНОЗИРОВАНИЯ ОЖИДАЕМЫХ ГОДОВЫХ МЕДИЦИНСКИХ РАСХОДОВ ПАЦИЕНТА ДЛЯ СТРАХОВОГО АНАЛИТИКА

Команда №6



Наша команда:

- Дмитрий - **Team Lead** - руководит проектом, определяет архитектуру, наводит на нужный путь и делает ревью.
- Валентина - **Data Scientist** - проводит аналитику данных, готовит датасет для ML, строит модели ML.
- Артур - **Backend** - разрабатывает бэкенд, отвечающий за ML-инференс, бизнес-логику и работу с БД.
- Илья - **Frontend** - разрабатывает интерфейс прототипа: формы ввода, отображение результатов и навигацию по приложению.
- Алёна - **QA** - проводит тестирование прототипа, оценивает качество системы.
- Матвей - **PM/BA** - отвечает за бизнес-метрики, представляет проект на отчётных занятиях.

Ссылки на участников:

- Дмитрий - <https://github.com/horacemtb>
- Валентина - <https://github.com/Valuxalo>
- Артур - <https://github.com/Archistein>
- Ильясия - <https://github.com/sultanovailsiya-rgb>
- Алёна - <https://github.com/al-lozinskaya>
- Матвей - <https://github.com/qwemiracle1337>

ТЕОРЕТИЧЕСКИЕ И ОРГАНИЗАЦИОННЫЕ ОСНОВЫ ПРОЕКТА:

- Актуальность проекта
- Постановка проблемы
- Цель и задачи проекта
- Основные этапы жизненного цикла цифрового продукта
- Объект и предмет исследования
- Практическая значимость проекта



Актуальность проекта:

- По данным РБК, объём рынка ДМС в России в 2025 году составил более 320 млрд руб., а сегмент корпоративного страхования сформировал около 127,8 млрд руб. премий по итогам первого полугодия 2025 года.
- Дополнительным фактором, усиливающим актуальность задачи, является ожидаемый рост стоимости полисов на 7–15 % в 2026 году на фоне медицинской инфляции в 10–13 %.
- Подтверждение перспективности подобных решений можно найти и на уровне публичных российских кейсов. Например, Страховой Дом ВСК внедрил «Систему поддержки андеррайтинговых решений по ДМС» на базе машинного обучения, в результате чего точность андеррайтинга выросла, а время обработки запроса сократилось (подробнее об этом на слайде «Анализ конкурентов и альтернативных решений»)

Постановка проблемы:

Страховой аналитик при оценке ожидаемых медицинских расходов клиента вынужден опираться на статичные таблицы коэффициентов и собственный экспертный опыт, что приводит к двум типовым ошибкам ценообразования:

- Ошибка первого рода (переоценка риска) ведёт к завышенному тарифу и оттоку «здоровых» клиентов к конкурентам, что снижает потенциальную прибыль компании.
- Ошибка второго рода (недооценка риска) приводит к назначению заниженной премии для пациентов с высокими ожидаемыми расходами, что напрямую увеличивает убыточность портфеля.



Цель и задачи проекта:

Цель проекта — разработать и протестировать прототип в виде веб-сервиса MedCost Prediction App для прогнозирования годовых медицинских расходов пациента и предоставления страховому аналитику инструментов поддержки принятия решений на основе методов машинного обучения.



Основные этапы жизненного цикла цифрового продукта:

- фаза Discovery — анализ рынка ДМС, исследование более целевой аудитории, изучение бизнес-кейсов конкурентов;
- фаза Design — формирование концепции продукта, проектирование пользовательских сценариев, создание макета интерфейса;
- фаза ML Research — построение и сравнение ML-моделей на синтетическом и расширенном датасетах, выбор финальной модели, расчёт метрик качества;
- фаза Delivery — разработка прототипа, реализация backend, frontend, OCR-подсистемы и контейнеризация решения;
- фаза Validation — тестирование готового прототипа, расчёт операционных, архитектурных и ML-метрик, подготовка дорожной карты дальнейшего развития.

Объект и предмет исследования:

- Объектом исследования являются процессы оценки медицинских рисков и расчёта страховой премии по полисам добровольного медицинского страхования в работе страховых аналитиков (андеррайтеров).
- Предметом исследования выступает программный сервис, использующий методы машинного обучения и табличные данные о пациенте для прогнозирования ожидаемых годовых медицинских расходов, а также для предоставления объяснения полученного результата на основе анализа факторов риска.

Практическая значимость проекта:

С прикладной точки зрения, реализованный программный продукт решает реальную бизнес-проблему страховых компаний:

- Позволяет сократить время оценки клиента с одного часа до десяти минут.
- Повысить обоснованность тарифа за счёт прозрачной аналитики факторов риска и снизить долю ошибочных андеррайтинговых решений за счёт перехода от субъективной экспертной оценки к статистически обоснованному прогнозу.
- Разработанное решение позволяет добиться до 83 % снижения временных затрат на обработку одного кейса и формирует экономию порядка 9,75 млн руб. в месяц для условной компании среднего размера с 60 аналитиками и потоком 36 000 случаев в месяц.

ИССЛЕДОВАНИЕ ПРЕДМЕТНОЙ ОБЛАСТИ, РЫНКА И ЦЕЛЕВОЙ АУДИТОРИИ:

- Характеристика предметной области
- Анализ целевой аудитории
- Анализ конкурентов и альтернативных решений
- Формулировка продуктовой гипотезы
- Методы исследования



Характеристика предметной области

- Предметная область проекта — добровольное медицинское страхование (ДМС) и связанный с ним процесс андеррайтинга, то есть оценки страхового риска и расчёта стоимости полиса для конкретного клиента.
- Базовой количественной метрикой качества андеррайтинга является коэффициент убыточности (loss ratio), рассчитываемый как отношение суммы страховых выплат и сопутствующих расходов на их регулирование к сумме заработанных страховых премий.
- С точки зрения операционной среды, страховой аналитик ежедневно работает в условиях высокого потока заявок: типовая компания среднего размера обрабатывает порядка 18 000–40 000 кейсов в месяц.

Анализ целевой аудитории:

В рамках Customer Development целевая аудитория продукта была разделена на основную, дополнительную и стратегическую группы. Это позволяет одновременно учитывать потребности конечного пользователя (того, кто будет ежедневно работать в системе) и потребности тех, кто будет принимать решение о её внедрении.

- *Основная целевая аудитория — страховой аналитик (андеррайтер)*
- *Дополнительная целевая аудитория — страховой агент и менеджер по продажам*



Анализ конкурентов и альтернативных решений:

Команда провела бенчмаркинг по двум направлениям: российские специализированные решения для андеррайтинга и международные аналоги. Цель — определить, насколько ниша заполнена, какие функции считаются стандартом отрасли и где у нового продукта существует пространство для дифференциации.



Анализ конкурентов и альтернативных решений:

Страховой Дом ВСК + MAINS Lab (Россия)

Внедрённая система поддержки андеррайтинговых решений по ДМС прогнозирует обращаемость корпоративного клиента и стоимость визита в клинику, учитывая сотни факторов. По данным открытых источников, точность андеррайтинга выросла в 1,5–1,7 раза, около 40 % обращений по ДМС обрабатываются автоматически, а время ответа сократилось в 15 раз.

- Основной плюс — это собственная разработка крупного страховщика с подтверждённым эффектом.
- Основные минусы — продукт не доступен другим компаниям как отдельный сервис и публично описан только в формате пресс-релизов 2022 года.

Анализ конкурентов и альтернативных решений:

Webiomed (Россия)

Платформа прогнозной аналитики и управления рисками для здравоохранения. Первая в России ИИ-система, зарегистрированная Росздравнадзором как программное медицинское изделие. Платформа умеет выявлять более 40 подозрений на заболевания, прогнозировать развитие более чем 10 ключевых проблем здоровья и работать с неструктурированными медицинскими записями через NLP. Заявленный экономический эффект — 7,1 млрд руб., а ориентир оценивается в 277 млн руб. на 1 млн населения в год.

- Основной плюс — клинически обоснованные модели и готовность к работе со страховщиками.
- Основной минус — продукт ориентирован прежде всего на клинические риски, а не на финансовое прогнозирование, и обладает высокой стоимостью внедрения.

Анализ конкурентов и альтернативных решений:

Ключевые инсайты по итогам анализа конкурентов:

- Российский рынок специализированных решений для андеррайтинга ДМС крайне узок: де-факто доступен только Webiomed, остальные решения являются внутренними разработками крупных страховщиков.
- У всех решений-аналогов есть высокий порог входа: высокая цена, фокус на корпоративные продажи или географические ограничения. Это формирует свободную нишу для лёгкого SaaS-продукта, ориентированного на малые и средние страховые компании.
- Объяснимость прогноза (SHAP, факторы риска) уже воспринимается рынком как обязательное требование, поэтому она должна быть включена в прототип с первой версии.
- Существенным дифференциатором продукта может стать наличие OCR-подсистемы, ускоряющей ввод данных пациента и приближающей пользовательский сценарий к реальной работе аналитика с бумажной анкетой.

Формулировка продуктовой гипотезы:

На основе анализа рынка, целевой аудитории и конкурентов командой были сформулированы две ключевые продуктовые гипотезы:

- **H1 (Гипотеза ценности):** если предоставить страховому аналитику веб-сервис, который по табличным данным о пациенте возвращает числовой прогноз ожидаемых годовых медицинских расходов и список ключевых факторов риска, время оценки одного клиента сократится с 60 минут до 10 минут, а точность тарификации повысится за счёт снижения доли ошибок переоценки и недооценки риска.
- **H2 (Гипотеза точности модели):** качество прогнозирования медицинских расходов методами машинного обучения (в первую очередь градиентного бустинга) на табличных данных будет существенно выше точности линейных моделей и экспертных таблиц коэффициентов, что подтверждается ML-метриками (MAE, RMSE, R^2 , MAPE) на отложенной тестовой выборке.

Методы исследования:

- Кабинетное исследование (desk research) — анализ открытых публикаций по объёму рынка ДМС, динамике loss ratio, публичных кейсов BCK, Webiomed и Merit Medicine, а также публикаций по применению ML-методов к прогнозированию медицинских расходов (Orji & Ukwandu, и др.).
- Сравнительный анализ конкурентов (бенчмаркинг) — структурированное сравнение функциональных возможностей существующих решений по семи критериям, включая объяснимость прогноза, доступность для малых и средних компаний и наличие OCR.
- ML-исследование на двух датасетах — основном синтетическом (5 000 наблюдений, 20 признаков) и расширенном (100 000 наблюдений, 50+ признаков).
- Экспериментальная проверка операционных и архитектурных метрик — измерение latency, throughput, времени распознавания OCR, расчёт OPEX и ROI на типовом профиле компании.

ПРОЕКТИРОВАНИЕ РЕШЕНИЯ:

- Общая концепция продукта
- Сценарии использования продукта (Use Cases)
- Требования к системе
- Создание прототипа



Общая концепция продукта:

Базовая user story проекта была сформулирована следующим образом:

«Как страховой аналитик, я хочу ввести основные параметры пациента в одну систему и получить числовой прогноз его медицинских расходов на следующий год для экономии времени».

Общая концепция продукта:

Конкурентные преимущества продукта, выявленные на этапе анализа рынка, формализованы в концепции в виде следующих принципов:

- фокус на индивидуальное страхование и сегмент малых и средних страховщиков, в отличие от корпоративных решений конкурентов;
- простой и быстрый интерфейс ввода данных с поддержкой OCR-распознавания бумажных анкет;
- встроенная объяснимость прогноза через SHAP — аналитик видит не только число, но и причины, по которым модель пришла именно к этому результату;
- возможность экспорта PDF-отчёта по конкретному кейсу для внутренней документации и коммуникации с клиентом;

Сценарии использования продукта (Use Cases):

Для формализации пользовательских сценариев было выделено три основных и пять дополнительных вариантов использования продукта.

Для каждого сценария была построена диаграмма последовательности UML (sequence diagram), описывающая взаимодействие аналитика, веб-интерфейса, бэкенда, базы данных и специализированных модулей (ML, SHAP, OCR).

•



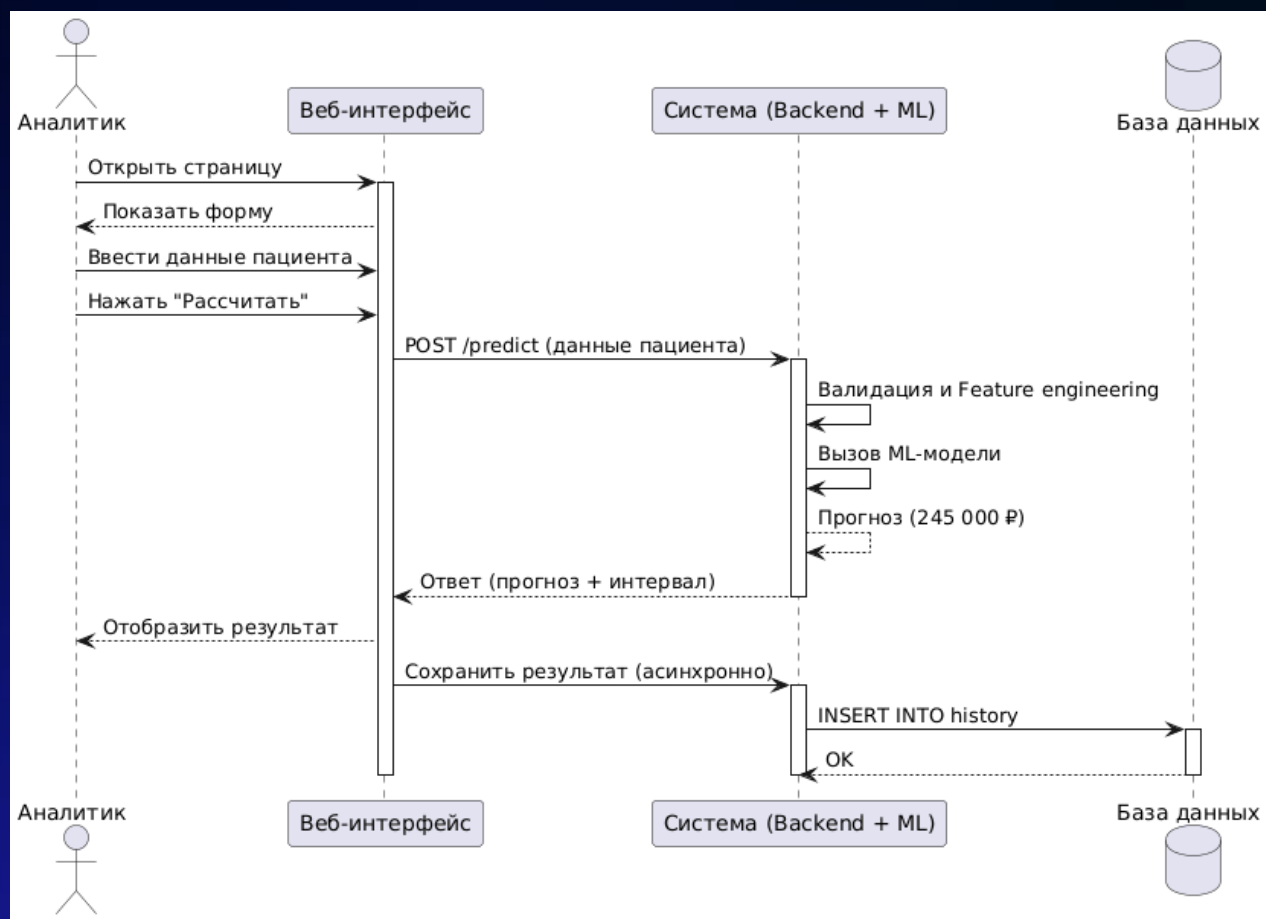
Сценарии использования продукта (Use Cases):

УС-1. Основной процесс расчёта прогноза

1. Аналитик открывает вкладку "Новый прогноз".
2. Веб-интерфейс загружает форму.
3. Аналитик заполняет поля (возраст, пол, ИМТ и т.д.) и нажимает кнопку "Рассчитать".
4. Веб-интерфейс отправляет данные (JSON) в Систему (Backend API).
5. Система (Backend) валидирует данные, обогащает их и вызывает ML-модель для инференса.
6. ML-модель возвращает рассчитанный прогноз (число).
7. Система (Backend) формирует ответ и отправляет его обратно во Веб-интерфейс.
8. Веб-интерфейс отображает результат на экране аналитика.
9. Одновременно Система (Backend) отправляет команду на сохранение (INSERT) в Базу данных.
10. База данных подтверждает сохранение записи.

Сценарии использования продукта (Use Cases):

УС-1. Основной процесс расчёта прогноза



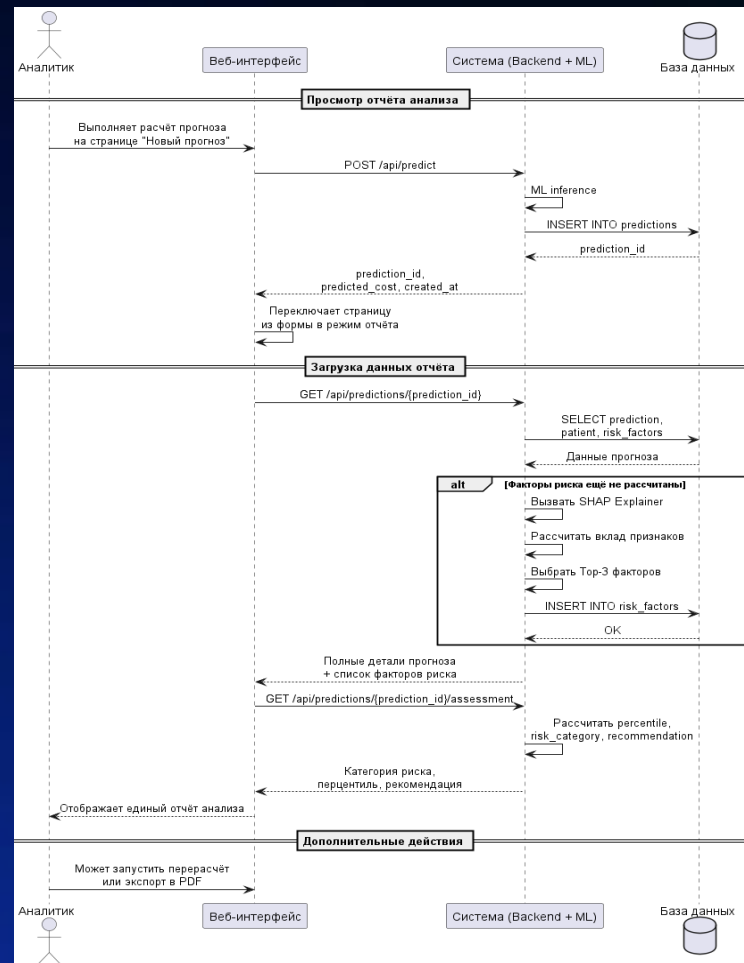
Сценарии использования продукта (Use Cases):

UC-2. Просмотр факторов риска (SHAP)

1. Аналитик получил результат расчёта (видит сумму 245 000 Р).
2. Аналитик нажимает на кнопку "Факторы риска".
3. Веб-интерфейс отправляет запрос к Backend на получение объяснения (SHAP values) для данного пациента.
4. SHAP-модуль возвращает список вкладов каждого признака (например: "Возраст: +45 000", "Диабет: +38 000").
5. Backend формирует ответ на Frontend.
6. Веб-интерфейс отображает их аналитику (обычно в виде списка или водопадной диаграммы).
7. Одновременно Backend отправляет команду на сохранение (INSERT) в Базу данных.
8. База данных подтверждает сохранение записи.

Сценарии использования продукта (Use Cases):

UC-2. Просмотр факторов риска (SHAP)



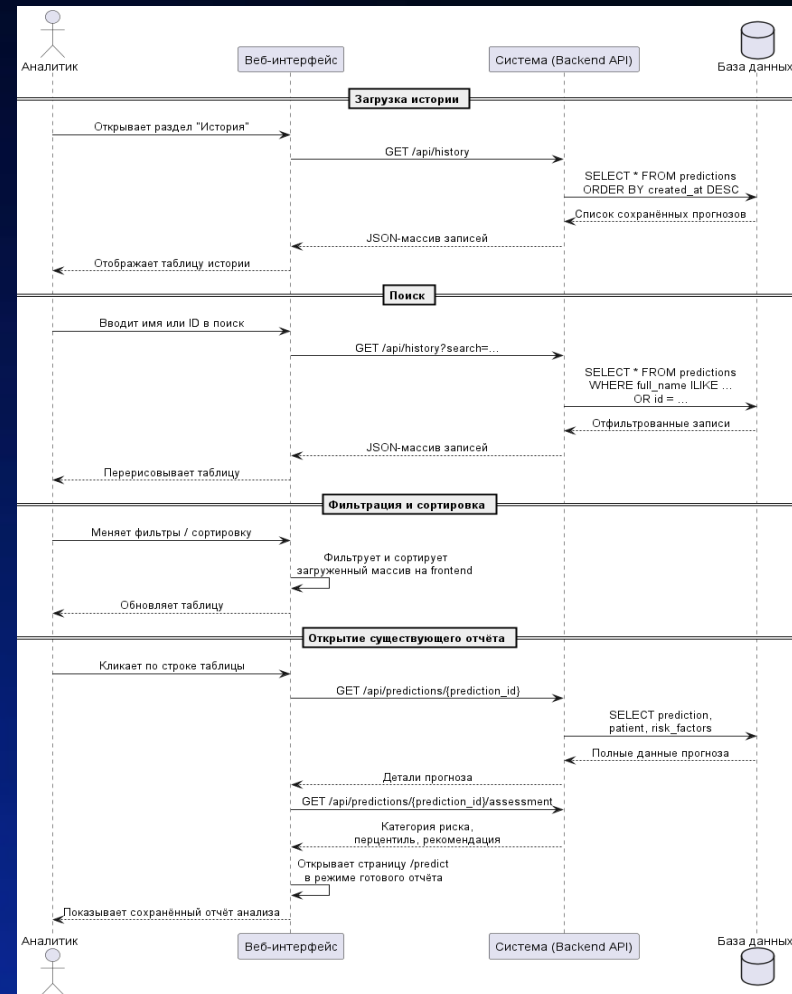
Сценарии использования продукта (Use Cases):

UC-3. История и поиск по прогнозам

1. Аналитик нажимает на кнопку "История" в меню.
2. Веб-интерфейс отправляет GET-запрос в Систему на получение списка (обычно с параметром пагинации).
3. Система обращается к БД.
4. БД возвращает набор записей (список пациентов с их параметрами, суммой и датой).
5. Система возвращает JSON-массив во Веб-интерфейс.
6. Веб-интерфейс отображает таблицу (ФИО, Возраст, Сумма, Дата).
7. Аналитик вводит текст в строку поиска (например, фамилию "Иванов" или его ID) и нажимает Enter.
8. Веб-интерфейс отправляет GET /history?search=Иванов в Систему.
9. Система выполняет SQL-запрос с условием WHERE name LIKE '%Иванов%'.
10. БД возвращает отфильтрованные данные.
11. Система возвращает их на Веб-интерфейс.
12. Веб-интерфейс перерисовывает таблицу, показывая только найденные записи.

Сценарии использования продукта (Use Cases):

UC-3. История и поиск по прогнозам



Сценарии использования продукта (Use Cases):

Дополнительные сценарии (UC-4 – UC-8):

- UC-4. Экспорт PDF-отчёта по конкретному прогнозу (ФИО, дата, прогноз, доверительный интервал, топ-3 фактора, логотип компании);
- UC-5. Удаление прогноза из истории с подтверждением через модальное окно и каскадным удалением связанных записей в `risk_factors`;
- UC-6. Сортировка таблицы истории по столбцам «Дата», «Прогноз» и «Возраст» с переключением направления сортировки;
- UC-7. Фильтрация записей в истории по диапазону стоимости и диапазону даты создания, с возможностью сброса всех фильтров;
- UC-8. Загрузка и OCR-распознавание бумажной анкеты пациента: пользователь выбирает файл изображения, бэкенд возвращает распознанные поля (JSON), и фронтенд автоматически заполняет ими форму;

Требования к системе:

Требования к системе разделены на:

- Функциональные
- Нефункциональные



Создание прототипа:

Прототип включает следующие ключевые экраны:

- вкладка «Новый прогноз» с формой ввода данных и кнопкой «Распознать анкету»;
- экран результата с числовым прогнозом, оценкой перцентиля относительно исторической выборки и кнопками для перехода к факторам риска и экспорту PDF;
- вкладка «История» с таблицей сохранённых прогнозов, строкой поиска, набором фильтров и сортировкой по столбцам;
- модальное окно подтверждения удаления;
- дашборд с агрегированной аналитикой по сохранённым прогнозам (распределение по полу, доля пациентов с факторами риска, гистограмма прогнозов);

РАЗРАБОТКА ПРОТОТИПА И ТЕХНИЧЕСКАЯ РЕАЛИЗАЦИЯ:

- Обоснование выбора технологического стека
- Архитектура решения
- CI/CD, контейнеризация и развертывание



Обоснование выбора технологического стека:

Backend

- Python 3.11
- FastAPI — реализация REST API.
- Pydantic — валидация входных и выходных схем.
- SQLAlchemy — ORM и работа с реляционной БД.
- Uvicorn — ASGI-сервер.
- python-multipart — загрузка файлов для OCR.

Почему выбран FastAPI:

- высокая скорость разработки;
- удобная декларативная валидация через Pydantic;
- встроенная OpenAPI/Swagger документация;
- простой и понятный способ описывать API для дальнейшего развития сервиса;

Обоснование выбора технологического стека:

Frontend

- React 18 — основа пользовательского интерфейса.
- TypeScript — типизация клиентской логики и API-контрактов.
- Vite — быстрый dev-сервер и сборка frontend-приложения.
- React Router — маршрутизация между разделами интерфейса.
- Tailwind CSS — утилитарная стилизация и адаптивный layout.
- Recharts — визуализация статистики и графиков на дашборде.
- Sonner — toast-уведомления для OCR, ошибок и пользовательских действий.

Почему выбран такой стек:

- фронтенд организован по модульной структуре ``app / features / pages / widgets / shared``;
- API-контракты типизированы и переиспользуются в клиентской логике;
- интерфейс собран из переиспользуемых виджетов и собственного UI-kit;

Обоснование выбора технологического стека:

База данных

В качестве СУБД выбрана PostgreSQL — надёжная реляционная база данных с поддержкой каскадных удалений, индексов по строковым полям (для поиска по ФИО) и широкой совместимостью с инструментами анализа данных.

Логическая модель данных приведена к третьей нормальной форме и включает четыре основные сущности:

- patients
- predictions
- risk_factors
- synthetic_cohort

Обоснование выбора технологического стека:

ML-стек

Для построения ML-модели использовались библиотеки scikit-learn, CatBoost, Optuna и SHAP. Финальная модель проекта — CatBoostRegressor, выбранная по результатам сравнительного исследования.

Для интерпретации прогноза применяется библиотека SHAP, для подбора гиперпараметров — комбинация GridSearchCV (для линейных моделей и Random Forest) и Optuna (для CatBoost и HistGradientBoosting).

Сохранение готовой модели выполняется через формат joblib (.joblib pipeline).



Обоснование выбора технологического стека:

OCR и обогащение данных

В проекте используется Tesseract OCR как базовый движок распознавания текста, потому что он хорошо подходит для обработки анкет фиксированного формата.

Основные причины выбора:

- open-source и бесплатность — Tesseract распространяется свободно и не требует оплаты за использование или зависимости от коммерческого OCR API;
- локальный запуск — Tesseract работает внутри backend-контейнера и не требует внешнего API;
- простая интеграция — его легко встроить в существующий Python/FastAPI backend;
- доступ к координатам текста — помимо распознанных слов можно получить bounding boxes и использовать их для привязки значений к полям анкеты;

CI/CD, контейнеризация и развертывание:

- Создан Dockerfile отдельно для backend и frontend, а также общий docker-compose.yml, объединяющий приложение, базу данных PostgreSQL и сервис frontend в единое изолированное окружение.
- Настроены переменные окружения через .env-файлы (с примером в .env.example и исключением .env из git): DATABASE_URL, MODEL_PATH, DADATA_API_KEY, DADATA_API_SECRET, VITE_API_BASE_URL.
- Реализована команда полного запуска проекта в одну строку: `docker compose up --build`. После запуска фронтенд доступен по адресу <http://localhost:8501>, бэкенд — <http://localhost:8000>, Swagger-документация — <http://localhost:8000/docs>.
- Настроены проверки на уровне GitHub Actions: при каждом пуше в ветку main запускается автоматическая сборка контейнеров и проверка линтерами (flake8, ESLint).

РАССЧЕТ ИТОГОВЫХ МЕТРИК:

- Метрики ML
- Метрики операционной эффективности
- Архитектурные метрики надежности
- Метрики Tesseract OCR
- Тестирование прототипа



Метрики ML:

В КА

- MAE - 37,09 (95 % CI [28,64; 49,94])
- RMSE - 176,75
- R^2 - 0,9993
- MAPE - 0,0018



Метрики ML:

Модель	MAE	RMSE	R ²	MAPE
CatBoost	35.26 [29.85; 44.13]	117.64 [42.76; 215.90]	0.9997 [0.9991; 1.0000]	0.0018 [0.0017; 0.0020]
CatBoost SHAP-selected	37.09 [28.64; 49.94]	176.75 [48.73; 330.49]	0.9993 [0.9979; 1.0000]	0.0018 [0.0016; 0.0021]
HistGradientBoosting	286.16 [256.78; 324.12]	591.25 [381.34; 863.46]	0.9929 [0.9860; 0.9970]	0.0151 [0.0141; 0.0162]
Random Forest	991.31 [938.70; 1050.68]	1334.38 [1246.41; 1437.86]	0.9654 [0.9606; 0.9695]	0.0569 [0.0537; 0.0604]
Best Linear ElasticNet	1083.21 [1011.97; 1158.43]	1598.57 [1421.91; 1797.68]	0.9503 [0.9406; 0.9584]	0.0550 [0.0524; 0.0576]

Главный результат ML исследования:

- Лучшими по качеству на test оказались две модели: CatBoostRegressor и CatBoostRegressor_SHAP_selected.
- По результатам paired bootstrap-сравнения по метрике MAE статистически значимой разницы между ними не обнаружено: 95% доверительный интервал для разницы MAE включает 0.
- На имеющейся тестовой выборке нельзя утверждать, что полная CatBoost-модель лучше сокращённой версии на основе SHAP-отбора признаков.
- С учётом сопоставимого качества и меньшего числа признаков в качестве финальной модели можно выбрать CatBoostRegressor_SHAP_selected как более компактную и интерпретируемую.



Метрики операционной эффективности:

- Time Reduction %
- Cost Saved per Month
- OPEX
- Cost per Inference
- Return on Investment (ROI)

Time Reduction %:

$\text{Time Reduction \%} = (\text{Базовое время} - \text{Новое время}) / \text{Базовое время} \times 100\%$

Базовые расходы (в среднем) = 60 мин

Новые расходы (в среднем) = 10 мин

Time Reduction % = 83% снижения затрат по времени

Cost Saved per Month:

Рассчитаем для средней компании и средней зарплате аналитика 100 000 руб/мес.

Cost Saved per Month = (Базовое время - Новое время) × Часовая ставка аналитика ×
Количество прогнозов

Cost Saved per Month = (50 мин/60)час × 650 руб/час × 18000 прогнозов/мес = 9 750 000
руб/мес

OPEX (Operating Expenses):

Стоимость аренды двух серверов в облаке с данными параметрами (Yandex Cloud) - 55 000 руб/мес.

Стоимость аренды PostgreSQL в облаке (Yandex Cloud) - 40 000 руб/мес.

Интеграция с dadata: подсказки 10000 запросов в день - бесплатно;
стандартизация - 7200 руб/мес.

Техническая поддержка (обновление, восстановление после сбоев, поддержка) - 50 000 руб/мес.

OPEX = 152 200 руб/мес.

Cost per Inference:

$\text{Cost per Inference} = (\text{Затраты на инфраструктуру модели за период}) /$
 $(\text{Количество инференсов за период})$

Затраты на инфраструктуру модели за период = 147 200 руб/мес.

Количество инференсов за период = 36 000 в мес

Cost per Inference = 4.1 руб.

Return on Investment (ROI):

$$\underline{ROI \text{ (месячный)}} = (\text{Cost Saved} - \text{OPEX}) / \text{OPEX} \times 100\%$$

$$\underline{ROI \text{ (месячный)}} = 6306 \%$$

Архитектурные метрики надежности:

- Latency
- Throughput
- API Error Rate
- Initial Load Time



Latency:

Время ответа системы на запрос расчёта предсказания прогноза:

Latency P95 = 57,237 мсек. (10000 запросов на локальный сервер)

Для удалённого сервера **Latency P95** $\approx$ 200 мсек.

Throughput:

Показатель того, сколько операций система обрабатывает в единицу времени:

Throughput = 14 прогнозов/сек.

API Error Rate (частота ошибок API):

Отношение числа неспешных запросов к числу успешных:

$$\textit{API Error Rate} = 0,03 \%$$

Initial Load Time (время начальной загрузки):

Initial Load Time P95 = 2,23 сек.

Метрики Tesseract OCR:

- Среднее время распознавания анкеты
- Accuracy
- CER (Character Error Rate)



Среднее время распознавания анкеты OCR:

Time OCR = 3,51 сек

Accuracy:

Точность для печатных документов (идеальные условия). Процент правильно распознанных символов или слов от их общего количества в эталонном тексте.

Accuracy = 95% - 98%

CER (Character Error Rate):

Частота ошибок на уровне отдельных символов. Считает количество вставленных, удаленных или замененных символов.

CER = 1.5% - 2.5% - для фото документов

Ссылка на документацию:

<https://github.com/horacemtb/medcost-prediction-app/blob/main/README.md>



Тестирование прототипа:

По итогам функционального тестирования все восемь пользовательских сценариев успешно проходят основные потоки. Зафиксированный Task Success Rate составил 100 %. Альтернативные потоки также обрабатывают корректно:

- некорректные данные приводят к ожидаемым HTTP 400
- отсутствие записи в БД — к HTTP 404 без падения интерфейса
- ошибки OCR — к подробному списку warnings
- ошибка PDF-генерации возвращает понятное пользователю сообщение.



Поставленные в начале проекта задачи выполнены:

- Проведён анализ рынка ДМС и сформировано экономическое обоснование
- Определена целевая аудитория продукта
- Выполнен бенчмаркинг конкурентов и зафиксирована свободная ниша
- Спроектированы архитектура системы, REST API из двенадцати эндпойнтов и логическая модель данных
- Проведено самостоятельное ML-исследование с сравнением десяти классов моделей
- Реализован прототип, включающий веб-интерфейс на React, бэкенд на FastAPI, базу данных PostgreSQL, OCR-подсистему на Tesseract и интеграцию с Dadata
- Проведено функциональное и нефункциональное тестирование, рассчитаны операционные, архитектурные и ML-метрики

Направления дальнейшего развития:

- переход от синтетического датасета к реальным данным страховой компании с пересборкой ML-модели и внедрением мониторинга дрейфа данных через PS
- реализация системы авторизации и ролевой модели (JWT-токены, разграничение ролей «аналитик» и «руководитель»)
- расширение feature set ML-модели за счёт перехода на расширенный датасет (100 000 наблюдений, 50+ признаков)
- интеграция продукта с корпоративными CRM, BI-системами и каналами поступления заявок страховых компаний
- развитие OCR-подсистемы для работы с произвольными форматами анкет (не только типовая форма)

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ:

- ГОСТ 7.32-2017. Система стандартов по информации, библиотечному и издательскому делу. Отчёт о научно-исследовательской работе. Структура и правила оформления. – Москва : Стандартинформ, 2017. – 27 с.
- ГОСТ Р 7.0.100-2018. Библиографическая запись. Библиографическое описание. Общие требования и правила составления. – Москва : Стандартинформ, 2018. – 124 с.
- ГОСТ Р 7.0.5-2008. Библиографическая ссылка. Общие требования и правила составления. – Москва : Стандартинформ, 2008. – 23 с.
- ВСК внедрила систему поддержки андеррайтинговых решений по ДМС на базе алгоритмов MAINS Lab [сайт] // Ведомости.— URL: https://www.vedomosti.ru/press_releases/2022/06/09/vsk-vnedrila-sistemu-podderzhki-anderraitingovih-reshenii-po-dms-na-baze-algo (дата обращения: 02.05.2026).

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ:

- Искусственный интеллект улучшает ДМС: ВСК сокращает срок обработки обращений по полисам медицинского страхования в 15 раз [сайт] // АСН — Агентство страховых новостей.
- Исследование методов распознавания текстовых документов с использованием компьютерного зрения [сайт] // ResearchGate. — URL: <https://www.researchgate.net/publication/380896842> (дата обращения: 13.05.2026).
- Перспективы цифрового андеррайтинга на основе машинного обучения и больших данных для страховых компаний [сайт] // Webiomed. — URL: <https://webiomed.ai/blog/perspektivy-tsifrovogo-anderraitinga-na-osnove-ML/> (дата обращения: 03.05.2026).

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ:

- Проблемы роста: как меняется рынок добровольного медицинского страхования [сайт] // РБК. – URL: <https://www.rbc.ru/industries/news/69a53c239a79470c79c41e18> (дата обращения: 25.04.2026).
- Цены на полисы ДМС выросли на 7-15% в 2026 году [сайт] // Коммерсантъ. – URL: <https://www.kommersant.ru/doc/8463315> (дата обращения: 28.04.2026).
- Чистая прибыль «Страховой Дом ВСК» по МСФО за 12 мес. 2025 г. снизилась на 40.38% до 6,64 млрд рублей [сайт] // Cbonds. – URL: <https://cbonds.ru/news/3892209/> (дата обращения: 03.05.2026).